Actividad 12
Tema 1. Almacenamiento de la información
Modulo: Bases de datos 26/27
Alumno: Maitane Navarro Carrasco

1) Investigue y explique con sus palabras en que consiste el método de búsqueda binaria en ficheros de datos.

Es un algoritmo usado para encontrar un elemento en una base de datos ordenada. La búsqueda no empieza desde el principio y vamos buscando registro a registro, si no que empieza desde la mitad del archivo.

Si el elemento que estamos buscando es menor que lo que encontramos en el medio de la base de datos, continuamos buscando en la primera mitad de la base de datos.

Por ejemplo supongamos que nuestra base de datos incluye los números:

10, 20, 30, 40, 50, 60, 70,80

Queremos encontrar el número 70 y comenzamos a buscar desde el medio, digamos 40.

La principal ventaja es que el elemento se puede encontrar con menos lecturas que en una búsqueda secuencial.

Como 70 es mayor no miramos los elementos anteriores y seguimos buscando en la segunda mitad hasta que tenemos el resultado.

Si el elemento buscado es mayor seguimos buscando en la segunda mitad.

Pero para que esto suceda la base de datos tiene que estar ordenada.

**********APUNTES JUAN CARLOS**********

A continuación, te explico en qué consiste cuando se aplica a un fichero (archivo) de datos:

Proceso del método de búsqueda binaria en un fichero

a) Requisito inicial

El fichero debe estar **ordenado** de manera ascendente o descendente con respecto al valor por el cual se quiere realizar la búsqueda.

b) Lectura del fichero.

A diferencia de la búsqueda binaria en una estructura de datos en memoria (como un arreglo o una lista), en un fichero no es práctico cargar todo el contenido en memoria si es grande. En lugar de eso, se leerán partes del fichero, según sea necesario, para ahorrar memoria.

c) Definir los límites iniciales:

i) Se establece un índice o posición inicial al principio del fichero (generalmente 0).

ii) Se establece un índice o posición final al final del fichero (determinado por el tamaño del fichero).

d) Dividir el fichero:

i) Se calcula el punto medio entre los índices inicial y final.

Esto se hace con la fórmula:

(Posición inicial + posición final) / 2

ii) Se busca el valor que está en esa posición media leyendo la parte correspondiente del fichero.

e) Comparación:

i) Si el valor en la posición media es igual al valor que estamos buscando, la búsqueda termina y hemos encontrado el elemento.

ii) Si el valor en la posición media es **mayor** que el valor que estamos buscando, se descarta la mitad superior del fichero (todo lo que esté por encima del valor medio), ajustando el límite final al valor inmediatamente anterior al medio.

iii) Si el valor en la posición media es **menor**, se descarta la mitad inferior del fichero, ajustando el límite inicial al valor inmediatamente posterior al medio.

f) Repetir el proceso:

El proceso de dividir y comparar se repite hasta que se encuentre el valor o hasta que los límites inicial y final se crucen, lo que indicaría que el valor no está presente en el fichero.

Ventajas:

i) Es muy eficiente, con una **complejidad temporal** de logaritmo en base 2 de n, donde n es el número de bloques en el fichero.

ii) Solo es necesario acceder a partes específicas del fichero, lo que es útil para trabajar con ficheros grandes.

Limitaciones:

iii) El fichero debe estar **ordenado** previamente, lo que puede requerir un procesamiento adicional si no es el caso.

iv) No es útil para ficheros desordenados o si se requiere un acceso secuencial a los datos.

En resumen, la búsqueda binaria en un fichero sigue los mismos principios que en estructuras de datos en memoria, pero se adapta al manejo de archivos mediante la lectura eficiente de partes del fichero para encontrar el valor objetivo.

2) Realice un ejemplo de búsqueda secuencial y binaria en clase suponiendo que tiene que acceder a un valor dentro de un conjunto ordenado de valores. Compute y compare el número de lecturas en ambos procesos para varios valores de búsqueda.

Para comparar los dos métodos vamos a utilizar el siguiente conjunto de valores ordenados:

10, 20, 30, 40, 50, 60, 70, 80, 90, 100

- **Búsqueda secuencial**

En la búsqueda secuencial se empieza por el primer valor y se van comprobando los siguientes hasta encontrar el que buscamos. Por ejemplo, si buscamos el valor **70**, tendríamos que hacer:

10 → 20 → 30 → 40 → 50 → 60 → **70** En este caso necesitamos **7 lecturas**.

Si buscamos el **20**, necesitamos 2 lecturas: 10 → **20**

Y si buscamos el **100**, tenemos que recorrer todos los valores, por lo que necesitamos **10 lecturas**.

- **Búsqueda binaria**

En la búsqueda binaria empezamos por la mitad y vamos descartando la parte que no nos interesa.

Por ejemplo, para buscar el **70**:

1. Miramos el valor central, que sería aproximadamente el **50**.
2. Como 70 es mayor que 50, descartamos los valores anteriores.
3. Miramos la mitad que queda y encontramos valores como 80.

4. Como 70 es menor que 80, volvemos a buscar en la parte anterior.

5. Finalmente encontramos el **70**.

En este caso necesitamos aproximadamente **3 lecturas**.

Podemos comparar algunos casos:

Valor buscado	**Búsqueda secuencial**	**Búsqueda binaria**
20	2 lecturas	3 lecturas aprox.
50	5 lecturas	1 lectura
70	7 lecturas	3 lecturas aprox.
100	10 lecturas	4 lecturas aprox.

Como podemos observar, la búsqueda secuencial es más sencilla, pero cuando tenemos muchos datos puede necesitar bastantes lecturas. La búsqueda binaria suele necesitar menos porque va descartando la mitad de los valores en cada paso.

3) ¿Cuántos índices primarios y de agrupamiento puede tener un fichero ordenado?

Índices primarios, solo podríamos tener uno, el fichero índice cuyo valor de indexación es la clave principal.
Índices de agrupamientos: de cero a varios.

4) Si tenemos un archivo de datos de 2.000 jugadores con tamaño fijo de bytes y un disco de tamaño de bloque igual a 1.024 bytes, determine el número de bloques requerido y el coste de una búsqueda binaria en cuanto a número necesario de accesos a bloques para encontrar un registro de datos.

Tenemos un archivo con **2.000 jugadores**, donde cada registro ocupa **80 bytes** y el tamaño de cada bloque es de **1.024 bytes.**

Primero calculamos cuántos registros caben en cada bloque:

1.024 / 80 = 12,8 = 12 registros

Como no podemos guardar una parte de un registro, caben **12 registros por bloque.**

Ahora calculamos los bloques necesarios:

2.000 / 12 = 166,66 = 167 bloques

Como no podemos tener una parte de un bloque, tenemos que redondear hacia arriba.

Por tanto, necesitamos:

167 bloques

- En que bloque estará el registro 1540?
  1540/12 = 128,333 = 129 Bloque
- Qué posición ocupa el registro 1540 dentro de su bloque?
  1540 12 = 4
- Coste de una búsqueda binaria en cuanto a número necesario de accesos a bloques para encontrar un registro de datos log⌈ 2 b⌉
  log2 167 = 7.38 → 8

5) Suponga que en el ejercicio anterior creamos un índice formado por la clave primaria (5 bytes) y un apuntador de 4 bytes. ¿Cuántas entradas de índice tendremos? ¿Cuantos accesos a bloques de disco necesitaremos ahora para efectuar una búsqueda binaria?

En el ejercicio anterior vamos a crear un índice formado por una **clave primaria de 5 bytes** y un **apuntador de 4 bytes.** Cada entrada del índice ocupa:

5 + 4 = 9 bytes

Como tenemos 2.000 jugadores, tendremos una entrada de índice por cada registro:

2.000 entradas de índice.

Ahora calculamos cuántas entradas caben en un bloque:

1.024 / 9 = 113,77 = 113

Por lo tanto, podemos guardar 113 entradas por bloque.

Calculamos los bloques que ocupa el índice:

2.000 / 113 = 17,69 = 18 bloques

Redondeando hacia arriba necesitamos:

18 bloques para el índice.

Al realizar una búsqueda binaria entre esos 18 bloques:

$\log_2(18) = 4.17 = 5$

Por tanto, necesitamos aproximadamente 5 accesos a bloques para localizar la entrada en el índice.